

The pencil corrections layer: operations, ingredients, and formulas

lgpsf dev notes

August 1, 2026

These notes are the mathematical companion to `dev/pencil-corrections-plan.md`. They explain, from the basics, what the corrections layer computes, from what ingredients, and by what formulas — and they explain the one deviation from the original plan (the mode block carries *two* coefficient matrices rather than one) by exhibiting the contradictions that forced it. Everything here is implemented in `include/lgpsf/corrections/` through the flip (slices 1–5 of the plan); deflation (slices 8–9) is included so the operation set is complete, and is marked as planned.

1 The problem and the ingredients

We have fitted a sparse, symmetric operator $\tilde{B} \in \mathbb{R}^{N \times N}$ that approximates a data-misfit Hessian H_d (for lgpsf, $\tilde{B}$ is the assembled fit with `Symmetrize::Weighted`; validated at field scale on the Pine Island Glacier ice-sheet Hessian). The consumer wants to solve regularized systems

$$(H_d + a H_r) x = b, \tag{1}$$

where H_r is their symmetric positive definite (SPD) regularization operator and $a > 0$ is a regularization parameter that *varies*: an L-curve sweep reuses one Hessian at many values of a , so every operation below takes a as an argument, and the build-time shift a_0 appears only in the contracts.

The fit $\tilde{B}$ is not directly usable in (1), for two reasons. First, it is indefinite: each row is fitted independently, so $\tilde{B}$ carries a tail of small spurious negative eigenvalues even though H_d is positive semidefinite. Second, it is inexact: the fit compresses $k \sim 100$ probe applications of H_d , and the residual error is significant in a predictable place (directions the probes resolved but the local model could not represent). The corrections layer repairs both — PSD-ification for the first, deflation for the second — and packages the result in two deployable forms (Section 2).

The layer is *operator-blind*: nothing in it reads a matrix entry. The complete list of ingredients is:

1. **apply $\tilde{B}$** — a caller-supplied symmetric operator, dimension plus block matvec (`SymmetricOp`);
2. **apply H_r** — cheap (mass and stiffness assemblies);
3. **solve H_r to a tolerance** — the consumer’s *oracle* (`HrOracle`), in practice Krylov preconditioned by multigrid; this is the expensive primitive, and every spectral operation costs $O(\#\text{modes})$ of them and nothing larger;

4. **the probe archive** — the pairs (Z, Y) with $Y = H_d Z$ that the fit was built from, plus any held-out and value-pass pairs; this is the *only* trace of H_d we hold;
5. **rank-sized dense algebra** — eigendecompositions and Cholesky factorizations of $\rho \times \rho$ matrices, $\rho \lesssim$ a few hundred.

Optionally the consumer supplies fresh applications of H_d itself, which unlocks the value pass. What we deliberately do *not* have: matrix entries, N -sized factorizations, or transpose matvecs (everything crossing the boundary is symmetric, and the layer verifies that stochastically at construction). The restriction is what makes the same machinery serve a sparse assembly, a block-low-rank format, or a matrix-free shell, and what makes a distributed-memory port a change of vector type rather than of algorithm: every N -sized object below is a tall-skinny block combined through rank-sized Gram matrices.

2 The two operators we deploy

All corrections are low-rank and live in one object, the *mode block* (Section 4). Writing E for the block’s correction and S for its *surrogate content* — the distinction is the subject of Section 7 — the layer deploys two different operators:

$$P(a) = \tilde{B} + E + a H_r, \quad (2)$$

$$M(a) = a H_r + S. \quad (3)$$

$P(a)$ is the *corrected operator itself*: the full accuracy of the sparse fit plus every correction, applied matrix-free. It is what one uses as a proposal precision or applies inside an inner Krylov iteration; its exact solve is iterative.

$M(a)$ is the *GLR deployment object*: shift plus low rank, so its inverse is closed-form (Section 4) — one oracle solve and $O(N\rho)$ arithmetic per application, at *every* a , with no refactorization. This is the preconditioner architecture validated end-to-end at PIG scale: the consumer’s Krylov iteration on (1) is preconditioned by $M(a)^{-1}$. The division of labor is worth stating plainly: the sparse fit is the information-compression device, and its pencil decomposition — obtainable from sparse applies and oracle solves alone — is the deployment device. $M(a)$ presents the part of the spectrum we have resolved; $P(a)$ retains everything the fit knows, resolved or not.

3 Why every spectral operation is a pencil operation

The typical H_d is an integral-kernel operator whose numerical rank r is fixed by the physics while the discretization dimension N grows under mesh refinement ($r \sim 10^3$ at $N \sim 6.5 \times 10^3$ for the PIG coarse mesh, the same r on refinements; $r \sim 10^4$ at $N \sim 10^6$ for a continental mesh). In the Euclidean spectrum of $\tilde{B}$ this limit is poison: the spectrum accumulates at 0 *from both sides*, with $N - r$ noise eigenvalues jitter-split around zero and no principled threshold between signal and noise. “Flip the negative eigenvalues of $\tilde{B}$ ” is an ill-posed request: it touches $\sim (N - r)/2$ modes, more on every refinement.

The generalized *pencil*

$$\tilde{B} v = \lambda H_r v \quad (4)$$

is the well-posed reformulation. Its eigenvalues are dimensionless data-to-prior ratios, mesh-independent, accumulating at 0 with only the physics-determined informed modes away from it; equivalently, they are the Euclidean eigenvalues of the prior-preconditioned operator $H_r^{-1/2} \tilde{B} H_r^{-1/2}$, computed without square roots. The design rule for the whole layer follows: *never enumerate modes near an accumulation point*. “The rightmost k pencil modes” and “every pencil mode below $-\gamma a_0$ ” are both requests for a physics-bounded, mesh-independent set; their Euclidean counterparts are not. Exact PSD of $\tilde{B}$ alone (the case $a_0 = 0$) is out of scope for the same reason: in the $N \rightarrow \infty$ limit it requires touching unboundedly many noise modes. Every contract below is relative to $(H_r, \gamma a_0)$.

4 The mode block, and the Woodbury identity that motivates it

Every spectral correction the layer makes is low-rank against H_r , and they all live in one object. The *mode block* holds an H_r -orthonormal basis $V \in \mathbb{R}^{N \times \rho}$,

$$V^T H_r V = I_\rho, \quad (5)$$

a provenance tag per column, and coefficient matrices over it. A correction term has the form

$$E = (H_r V) C (H_r V)^T \quad \text{with } C = C^T \in \mathbb{R}^{\rho \times \rho}. \quad (6)$$

The factors of H_r look odd at first sight; they are what makes the form close under the pencil. If (θ_i, u_i) are eigenpairs of C , then with $v_i = V u_i$ one checks directly, using (5), that

$$E v_i = \theta_i H_r v_i, \quad (7)$$

i.e. *the block’s pencil eigenvalues against H_r are exactly $\text{eig}(C)$* . Every positive-definiteness certificate in the layer is analytic because of this one fact.

The second payoff is the shifted inverse. Consider $M(a) = a H_r + (H_r V) C (H_r V)^T$ as in (3), and eigendecompose $C = U \Theta U^T$, $\Theta = \text{diag}(\theta_1, \dots, \theta_\rho)$. The Sherman–Morrison–Woodbury formula, with $W = H_r V$, needs the capacitance $C^{-1} + \frac{1}{a} W^T H_r^{-1} W$; but $W^T H_r^{-1} W = V^T H_r V = I$ by (5), so the capacitance is *diagonal* in the eigenbasis of C and the inverse closes over the oracle:

$$\boxed{M(a)^{-1} x = \frac{1}{a} H_r^{-1} x - V U \text{diag}\left(\frac{\theta_i}{a(a+\theta_i)}\right) U^T V^T x.} \quad (8)$$

One oracle solve plus $O(N\rho)$ arithmetic per application. The formula is easiest to believe in H_r -coordinates: on the eigendirection v_i the right side gives $\frac{1}{a} - \frac{\theta_i}{a(a+\theta_i)} = \frac{1}{a+\theta_i}$, which is the correct inverse of $a+\theta_i$; on the H_r -orthogonal complement of $\text{span}(V)$ it gives $\frac{1}{a} H_r^{-1}$, as required. Consequently (8) is exact for *every* symmetric C , invertible or not, and

$$M(a) \succ 0 \iff a > \max_i(0, -\min_i \theta_i), \quad (9)$$

an analytic certificate (`glr_pd_floor`). Changing a in (8) is scalar arithmetic — this is why an entire L-curve sweep pays for no factorization anywhere.

4.1 Merging contributions

Corrections arrive incrementally (a flip pass, then a deflation, then a deeper cache), so the block must absorb a new contribution $(H_r V_{\text{new}}) C_{\text{new}} (H_r V_{\text{new}})^T$, $V_{\text{new}} \in \mathbb{R}^{N \times q}$, without disturbing what is already there. Existing columns are never modified — provenance stays per-column, and previously computed quantities stay bitwise valid. The merge is classical Gram–Schmidt in the H_r inner product, done with Gram matrices so it costs q oracle *applies* (no solves) plus rank-sized dense algebra:

$$\begin{aligned} A &= V^T H_r V_{\text{new}} \in \mathbb{R}^{\rho \times q} && \text{(coefficients on existing columns),} \\ W &= V_{\text{new}} - V A && \text{(residual outside the block),} \\ G &= W^T H_r W = Q \Lambda Q^T && \text{(residual Gram, eigendecomposed),} \\ V_{\text{add}} &= W Q_+ \Lambda_+^{-1/2} && \text{(new } H_r\text{-orthonormal columns),} \end{aligned} \quad (10)$$

where $Q_+ \Lambda_+$ keeps the eigenvalues above a drop tolerance (relative to the largest candidate’s squared H_r -norm), and a second orthogonalization pass makes the arithmetic trustworthy. Candidates that lie (nearly) inside the current span produce no new columns; their contribution lands in the coefficients instead. With $B = V_{\text{add}}^T H_r V_{\text{new}}$ and $M = \begin{bmatrix} A \\ B \end{bmatrix}$, the extended coefficient matrix is

$$C \longleftarrow \begin{bmatrix} C & 0 \\ 0 & 0 \end{bmatrix} + M C_{\text{new}} M^T, \quad (11)$$

which is exact whenever nothing was dropped: it is just $V_{\text{new}} = V A + V_{\text{add}} B$ substituted into the contribution. Merging the same directions twice therefore adds coefficients, not columns.

5 The spectral machine: Lanczos in the H_r inner product

Both ends of the pencil spectrum come from one machine. The operator $L = H_r^{-1}(\tilde{B} + E)$ is self-adjoint in the inner product $\langle x, y \rangle_{H_r} = x^T H_r y$:

$$\langle Lx, y \rangle_{H_r} = x^T (\tilde{B} + E)^T H_r^{-1} H_r y = x^T (\tilde{B} + E) y = \langle x, Ly \rangle_{H_r}, \quad (12)$$

so Lanczos applies verbatim with $\langle \cdot, \cdot \rangle_{H_r}$ in place of the dot product. One iteration costs one operator apply, one oracle solve (applying L), and one H_r apply (for the norm); storing the images $H_r q_j$ makes every reorthogonalization projection oracle-free. We use full reorthogonalization against the Krylov basis *and against the block*: modes already in the block are invisible to the iteration, which is what makes every operation incremental — nothing is ever found twice, and a second call continues where the first stopped. For a Ritz pair $(\theta, y = Qs)$ from the tridiagonal T_m , the standard residual identity holds in the H_r -norm,

$$\|Ly - \theta y\|_{H_r} = \beta_m |s_m|, \quad (13)$$

so convergence is monitored for free. Two engine details found the hard way: on breakdown (an invariant subspace), the iteration restarts with a fresh random vector in the H_r -orthogonal complement — itself invariant, since L is self-adjoint — and a zero coupling in T_m decouples the finished block correctly; and the engine’s internal seed is salted, because a caller who built the operator’s data from the same innocently-chosen seed can otherwise hand the engine start

vectors correlated with the operator (in the adversarial case, every start vector is an exact eigenvector and every emptiness test lies).

`extend_modes` runs this machine and caches converged *rightmost* pairs (λ_i, v_i) in the block; the report carries the first value *not* cached, which is the λ_{k+1} that governs two-level inner conditioning. `make_pd` runs the same machine for the leftmost end, next.

6 PSD-ification: the a_0 -tied flip

Let (λ_i, v_i) be pencil eigenpairs (4) of the current operator, and write $w_i = H_r v_i$. With a safety factor $\gamma \in (0, 1)$ (default $\frac{1}{2}$), we correct exactly the modes below the threshold $-\gamma a_0$:

$$\tilde{B}_{\text{pd}} = \tilde{B} - c \sum_{\lambda_i < -\gamma a_0} \lambda_i w_i w_i^T, \quad c = \begin{cases} 2 & \text{(flip)} \\ 1 & \text{(relu)}, \end{cases} \quad (14)$$

implemented as block entries — the operator handle is never modified. Using $w_i w_i^T v_i = H_r v_i (v_i^T H_r v_i) = H_r v_i$, each corrected mode moves as

$$\lambda_i \mapsto (1 - c) \lambda_i = \begin{cases} -\lambda_i & \text{(flip)} \\ 0 & \text{(relu)}, \end{cases} \quad (15)$$

and every other mode is untouched. The slightly-negative noise tail above the threshold is deliberately left in place — the deployment shift absorbs it, and enumerating it is exactly the ill-posed operation of Section 3. What makes the operation a contract rather than a heuristic is the byproduct: the same Lanczos run resolves the leftmost *surviving* pencil value $\lambda_{\text{floor}} \in [-\gamma a_0, 0]$, and then, exactly,

$$\tilde{B}_{\text{pd}} + a H_r \succ 0 \iff a > -\lambda_{\text{floor}}, \quad \text{and} \quad \tilde{B}_{\text{pd}} + a_0 H_r \succeq (1 - \gamma) a_0 H_r. \quad (16)$$

Certification requires the leftmost surviving Ritz pair to *converge*: Ritz values interlace the true spectrum from above, so an unconverged leftmost estimate proves nothing. On budget exhaustion `make_pd` reports uncertified with progress kept — corrected modes stay in the block — and a later call continues. Flip versus relu differed by at most one PCG iteration at every depth measured on the PIG problem; flip is the default.

7 The deviation: one coefficient matrix cannot serve both operators

The original plan gave the block a single coefficient matrix C , used both in the correction E of (2) and in the surrogate S of (3). Working Section 5 exposed the flaw: the two deployment objects need *different numbers on the same directions*. Two concrete contradictions, each a one-line calculation.

Contradiction 1: caching a mode must not change the operator. For $M(a)$ to precondition well, the block must hold the operator's rightmost pencil modes: merge a converged eigenpair (λ, v) of $\tilde{B}$ with coefficient λ , so that S presents it. If that same coefficient also enters E , the represented operator becomes $\tilde{B} + \lambda H_r v v^T H_r$, and on the mode itself

$$(\tilde{B} + \lambda H_r v v^T H_r) v = \lambda H_r v + \lambda H_r v = 2\lambda H_r v : \quad (17)$$

caching has *doubled* the mode. The cache needs coefficient λ in S and 0 in E .

Contradiction 2: a flipped mode has two different values. The flip (14) needs coefficient $-c\lambda$ in E — that is the surgery that moves the operator’s eigenvalue to $(1 - c)\lambda$. But the surrogate must present the *corrected* operator’s content on that direction, which by (15) is $(1 - c)\lambda$. With one matrix, either the operator is wrong (using $(1 - c)\lambda$ under-corrects $\tilde{B}$) or the preconditioner is wrong ($M(a)$ presents the mode at $-c\lambda = -2\lambda$ instead of $-\lambda$; off by exactly λ).

The resolution is one basis, two coefficient matrices:

$$E = (H_r V) C_{\text{corr}} (H_r V)^T, \quad S = (H_r V) C_{\text{surr}} (H_r V)^T, \quad (18)$$

with C_{corr} the correction added to $\tilde{B}$ and C_{surr} the *known spectral content* of $\tilde{B} + E$ — what $M(a)$ deploys. Sharing the basis is right because the two matrices overlap heavily on the same directions (a flipped mode carries both numbers), and because the merge (10)–(11) extends both matrices through the same M . Each merge states both increments explicitly:

contribution	ΔC_{corr}	ΔC_{surr}
cache a pencil mode (λ, v) of $\tilde{B} + E$	0	λ
plain correction Δ (e.g. deflation)	Δ	Δ
flip/relu of a mode at λ	$-c\lambda$	$(1 - c)\lambda$

The consistency rule behind the table: whatever a merge adds to the operator is also known content of $\tilde{B} + E$ and belongs in C_{surr} ; *plus* whatever content of $\tilde{B}$ itself the caller has learned. A flip pass knows the mode’s $\tilde{B}$ -eigenvalue λ from the same Lanczos run, so its surrogate increment is $\lambda + (-c\lambda) = (1 - c)\lambda$ — the table’s last row is the first two rows composed. A deflation pass knows only its correction Δ , so on its directions C_{surr} holds Δ but not the (unresolved) content of $\tilde{B}$ there; that is an acknowledged approximation, and a later `extend_modes` on the corrected operator refines the surrogate exactly where it is weakest. The invariants are pinned by tests: a cache-only merge leaves $P(a)$ value-identical while moving the floor (9), and after a flip the surrogate presents the corrected values $-\lambda$, not -2λ .

With the split, the certificates separate cleanly: $\text{eig}(C_{\text{corr}})$ feeds the positive-definiteness reasoning for $P(a)$ (the flip contract (16) and, later, the warning-zone checks for $a < a_0$), while $\text{eig}(C_{\text{surr}})$ gives the exact floor (9) for $M(a)$.

8 Deflation (planned: slices 8–9)

For completeness, the remaining operations and their formulas; the observed numbers are from the maintainer’s PIG study.

Free deflation. The archive residuals $R = Y - \tilde{B}Z$ are exact samples of the error action $(H_d - \tilde{B})Z$, already paid for. In the build metric $M_0 = \tilde{B}_{\text{pd}} + a_0 H_r$, compute $M_0^{-1}R$ by the solve path, M_0 -orthonormalize it to $\tilde{Q}$ via the Cholesky factor of the Gram matrix $R^T M_0^{-1} R$ (square-root-free), and form the small matrix

$$T = \text{sym}\left((\tilde{Q}^T R) (\tilde{Q}^T M_0 Z)_{\text{rcond}}^+\right), \quad (19)$$

whose eigenpairs estimate the whitened error $M_0^{-1/2}(H_d - \tilde{B}_{pd})M_0^{-1/2}$ on the residual subspace. The truncated pseudoinverse is essential: with the raw pseudoinverse the estimated values reached 27.7 against a certified extreme of 3.10 and the corrected operator lost definiteness. Kept modes are clamped below at the clamp floor (default -0.95) and merged as a plain correction. The residual *subspace* is accurate (principal cosines 0.99 against certified directions); the *values* are the limiting factor — iteration counts improved $34 \rightarrow 22$ at the best setting and no setting helped at $k \leq 20$.

Value pass. If the consumer will spend m fresh applications of H_d , spend them on values, not directions: take the top- m left singular basis Q of the whitened residuals (Gram-based), compute $H_d Q$, and form the exact Rayleigh matrix of the whitened error on that basis. The new pairs $(Q, H_d Q)$ enter the archive as secant information, reusable by any later rebuild. On PIG this was decisive: $k=100$ fit plus a 50-apply value pass gave 5/12/18/24 iterations (to $10^{-2}/10^{-4}/10^{-6}/10^{-8}$) against the true operator, versus 12/26/36/45 undeflated, with zero clamp events.

9 Status

Implemented and tested through slice 5: the boundary (`symmetric_op.hpp`, `hr_oracle.hpp`), the two-matrix mode block (6)/(11) (`mode_block.hpp`), the struct with the Woodbury deployment (8)/(9) (`shifted_operator.hpp`), and the spectral machine with `extend_modes` and `make_pd` (14)–(16) (`pencil_lanczos.hpp`). Remaining: zone semantics for $a < a_0$, the two-level solve, deflation (19), the value pass, `rebuild.at`, the capability-gated Cholesky backend, and the maintainer-side validation runs. The plan of record is `dev/pencil-corrections-plan.md`; decisions are logged in `dev/design-notes.md`.